Tutoriel : création d'une application iOS avec deux View Controllers

Il existe plusieurs manières de passer d'un View Controller à un autre. Nous allons ici utiliser les transitions c'est-à-dire en bon franglais, les *segues*.

Création du projet Xcode

Comme précédemment, nous devons créer un projet Xcode. Cette fois nous l'appellerons `MultipleViewControllers Tuto`.

Création de l'interface

Sélectionnez le `Main.storyboard` à gauche, la silhouette d'un iPhone apparaît. Cliquez ensuite sur le + en à gauche de l'écran pour faire apparaitre la bibliothèque de *widgets* que vous pouvez ajouter. Tapez `controller` dans le champ `Objects` pour filtrer la recherche.

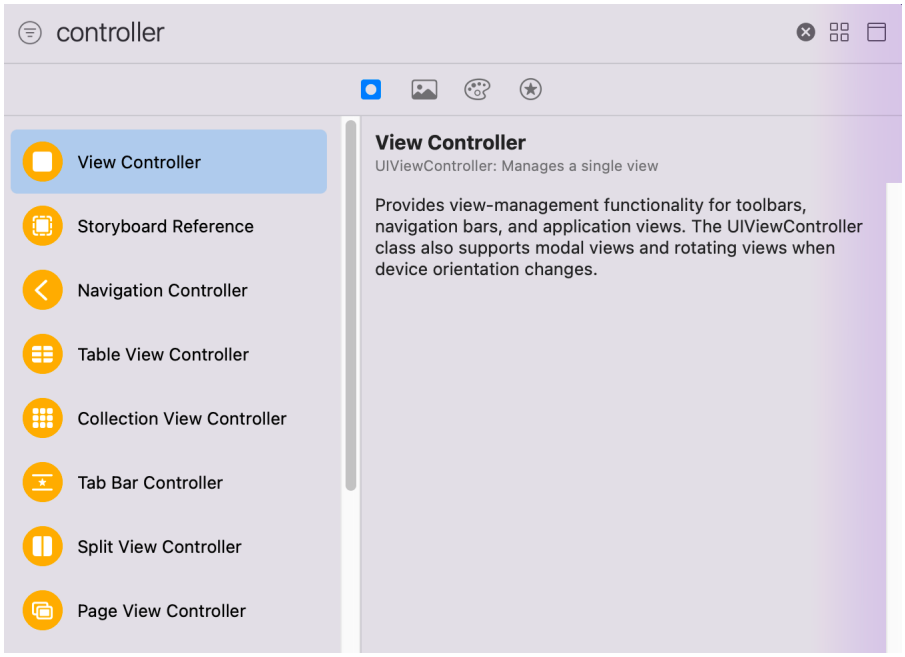


Figure 1Sélectionnez View Controller.

Sélectionnez `View Controller` et faites-le glisser à côté du `ViewController` actuel. On se retrouve avec deux View Controllers côte à côte.

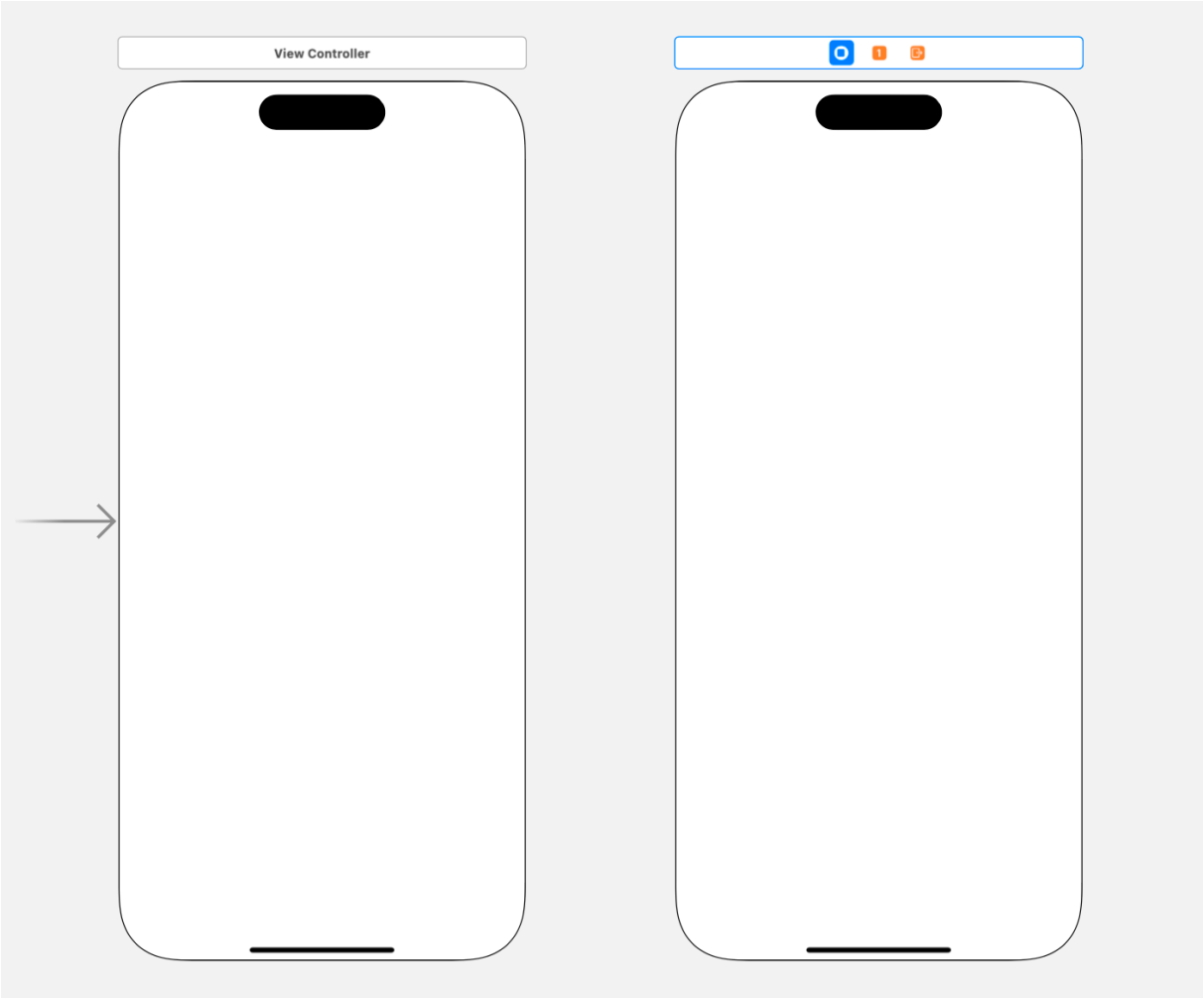
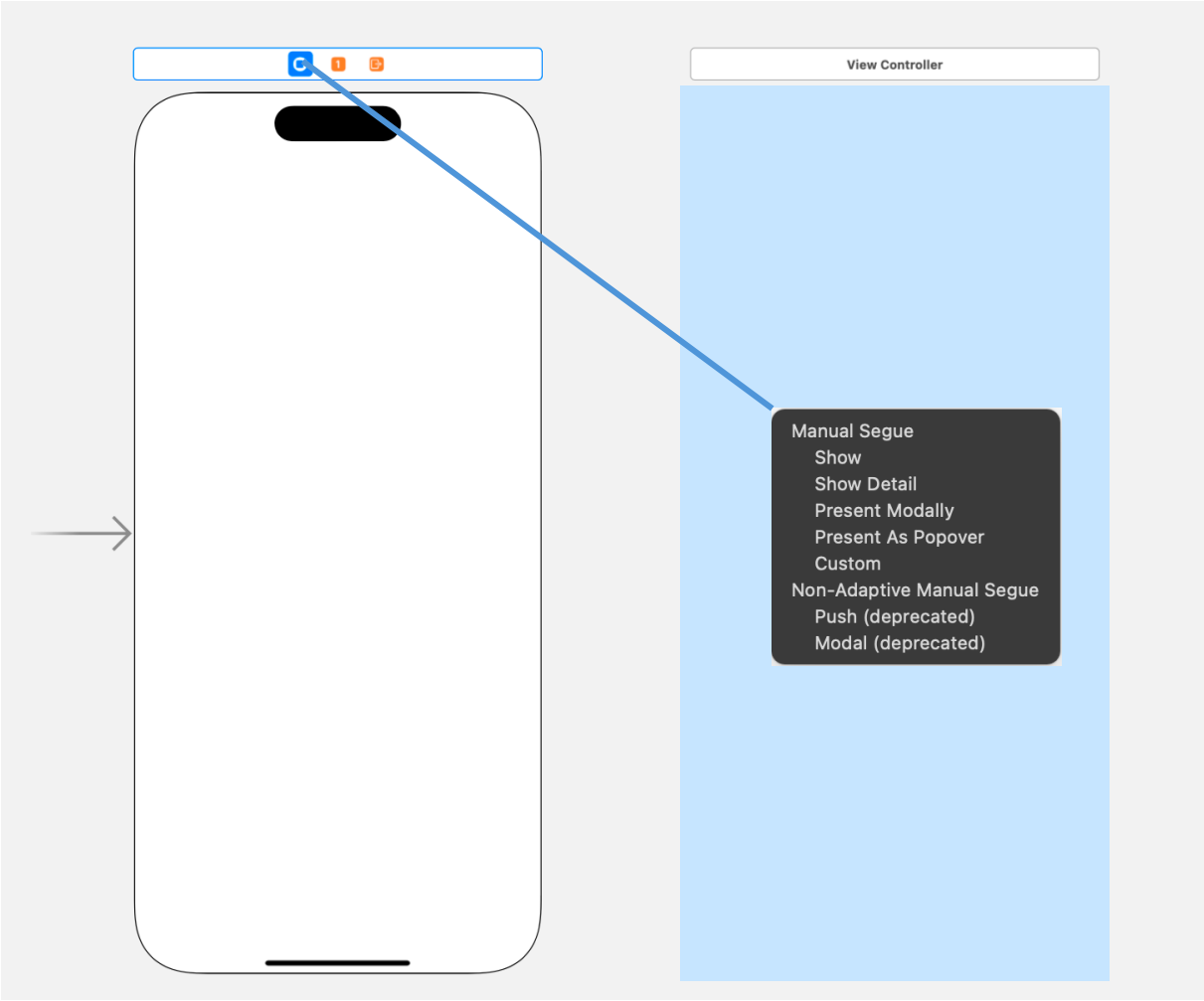
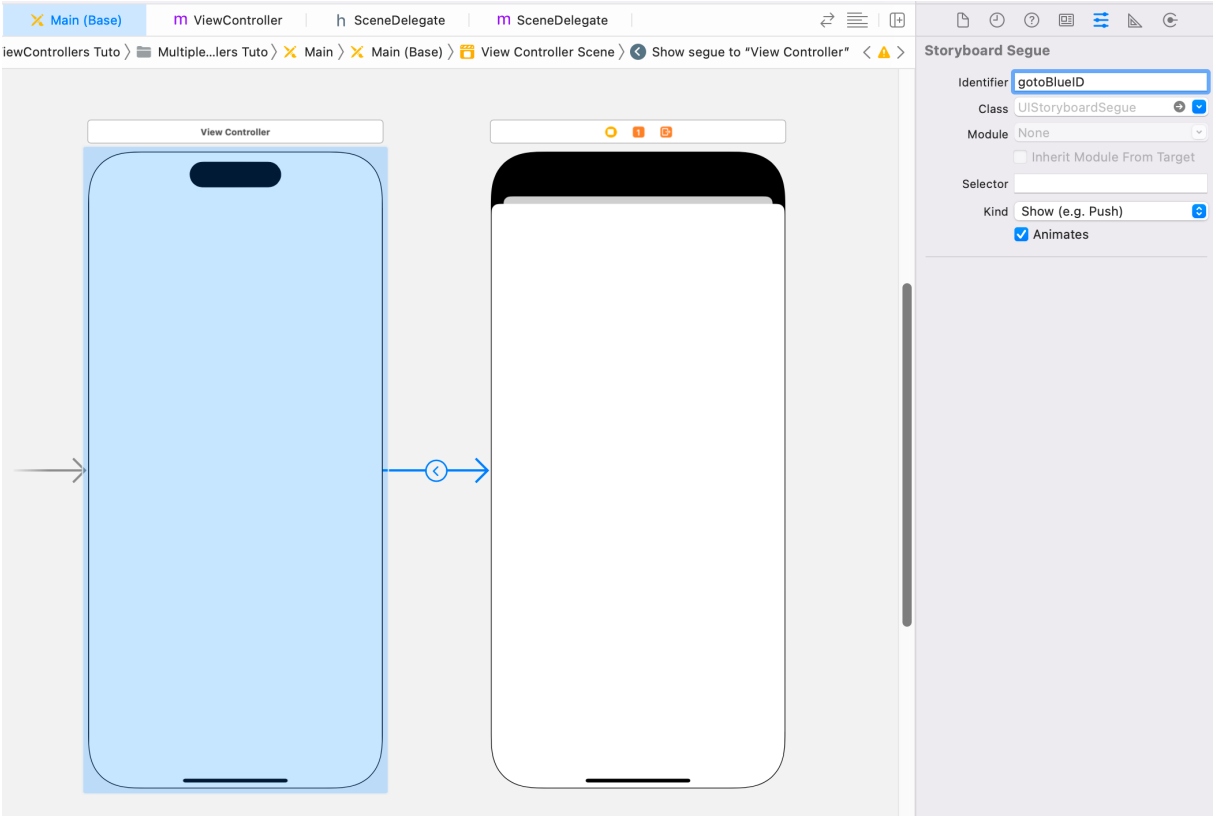


Figure 2Deux View Controllers.

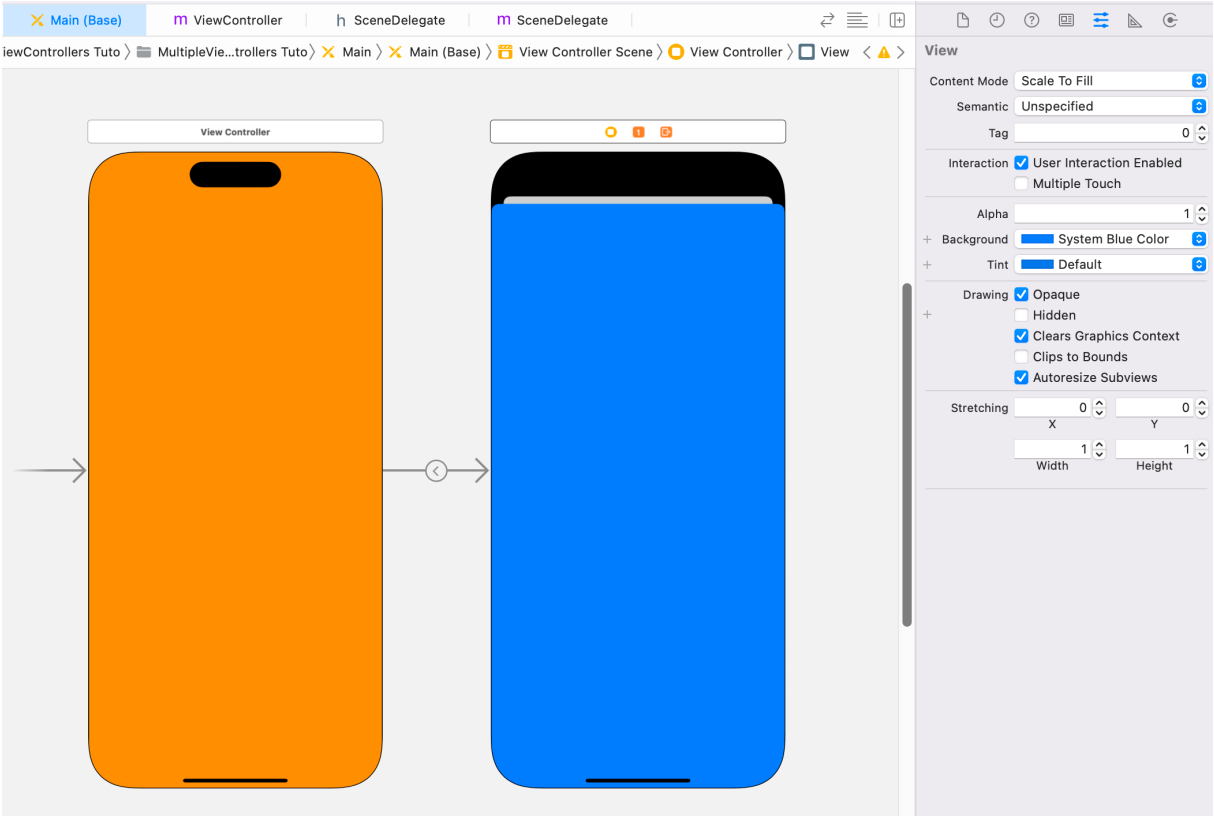
Nous allons maintenant créer un segue entre les deux. Pour cela cliquez sur le premier View Controller pour faire apparaître les trois icônes qui le surplombent ; sélectionnez le disque orange avec un carré blanc (il devient bleu), en maintenant la touche `control` glissez la souris jusqu'au deuxième View Controller en traçant un trait bleu ; le menu Segue apparait, sélectionnez Show.



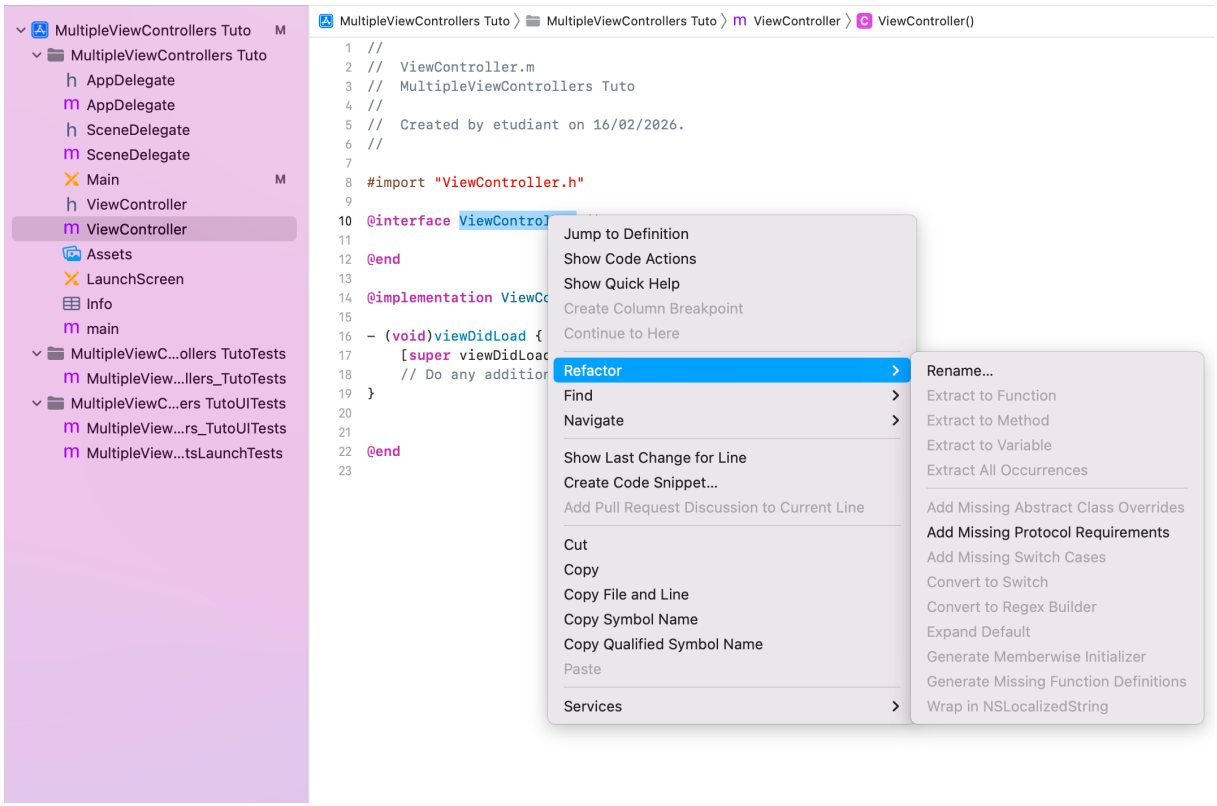
Nos deux View Controllers sont désormais liés par une transition segue. Sélectionnez cette flèche entre les deux View Controllers et dans l'`Attribute Inspector` donnez lui un identifiant (je propose `gotoBlueID`).

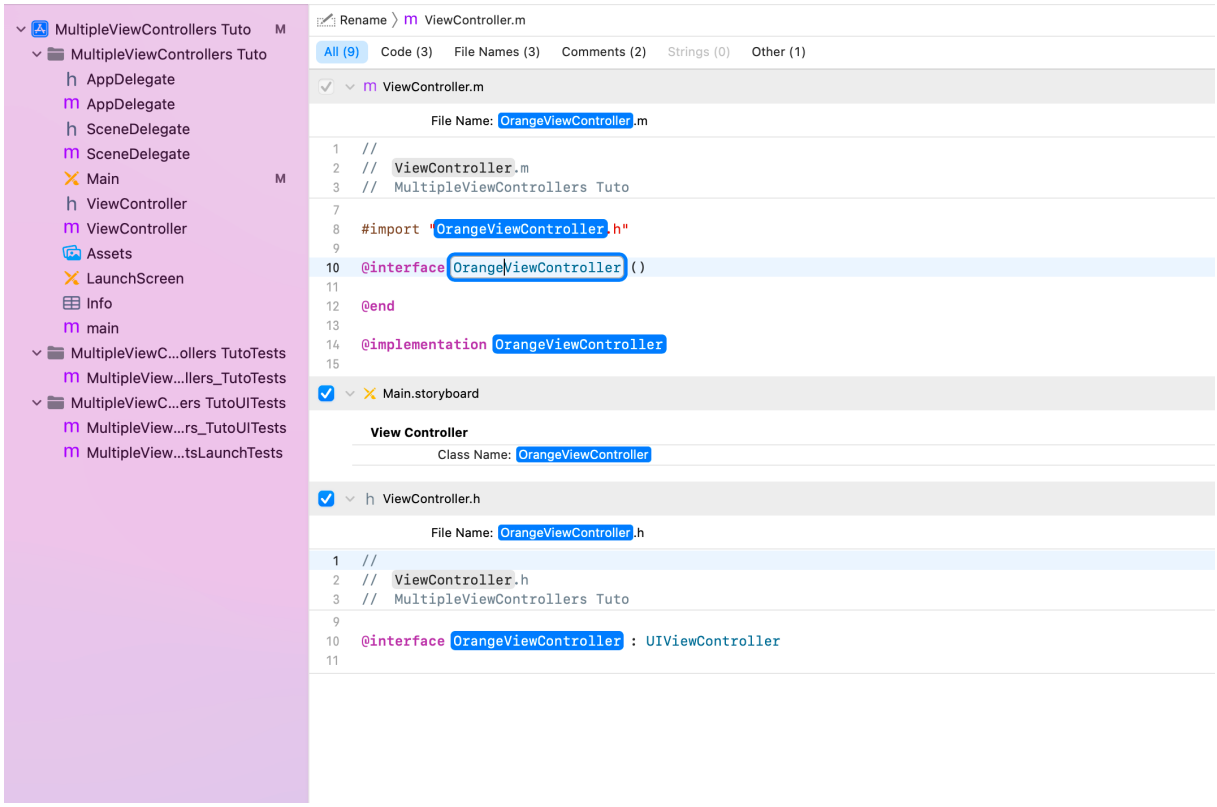


Comme nous avons deux View Controllers nous allons les différencier pour simplifier en changeant leur couleur de fond. Pour cela rien de plus simple, il suffit de sélectionner un View Controller et dans le panneau de droite en choisissant `l'Attribute Inspector`, de lui choisir une couleur. Pour l'exercice le View Controller de gauche aura la couleur `System Orange Color` et celui de droite (le nouveau) la couleur `System Blue Color`.



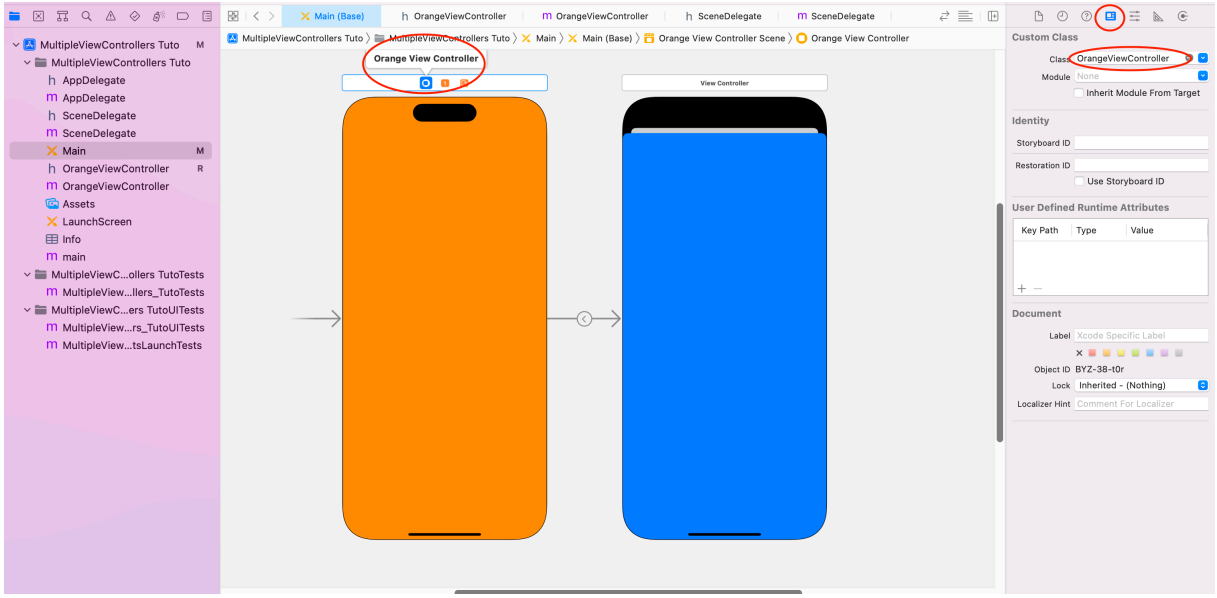
Puisque nous avons changé la couleur de notre premier View Controller, celui qui est géré initialement par la classe `ViewController` (définie dans les fichiers `ViewController.h` et `ViewController.m`), restons cohérent et renommons cette classe. Pour cela, il suffit d'aller dans l'un des deux fichiers (par exemple `ViewController.m`) de sélectionner le nom de la classe de faire un clic bouton droit dessus pour ouvrir un menu contextuel et de choisir `Refactor > Rename…` et de lui attribuer son nouveau nom, dans notre cas : `OrangeViewController`.



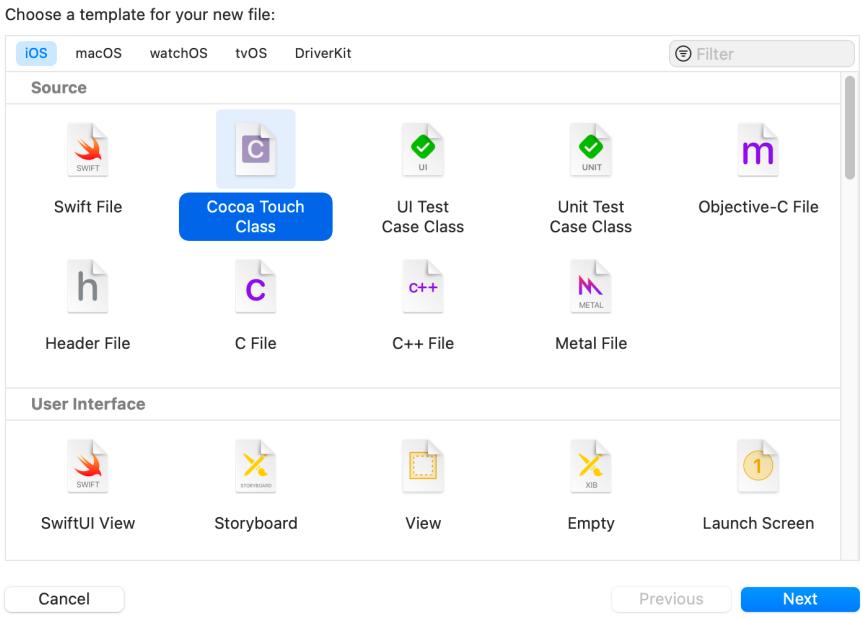


On peut remarquer que `ViewController` a été changé partout par `OrangeViewController`, dans le `.h` et le `.m` correspondants. On peut même constater, lorsqu'on sélectionne l'icône en forme de disque orange carré blanc, dans le `Main.storyboard`, et qu'on regarde dans le panneau latéral droit, l'`Identity`

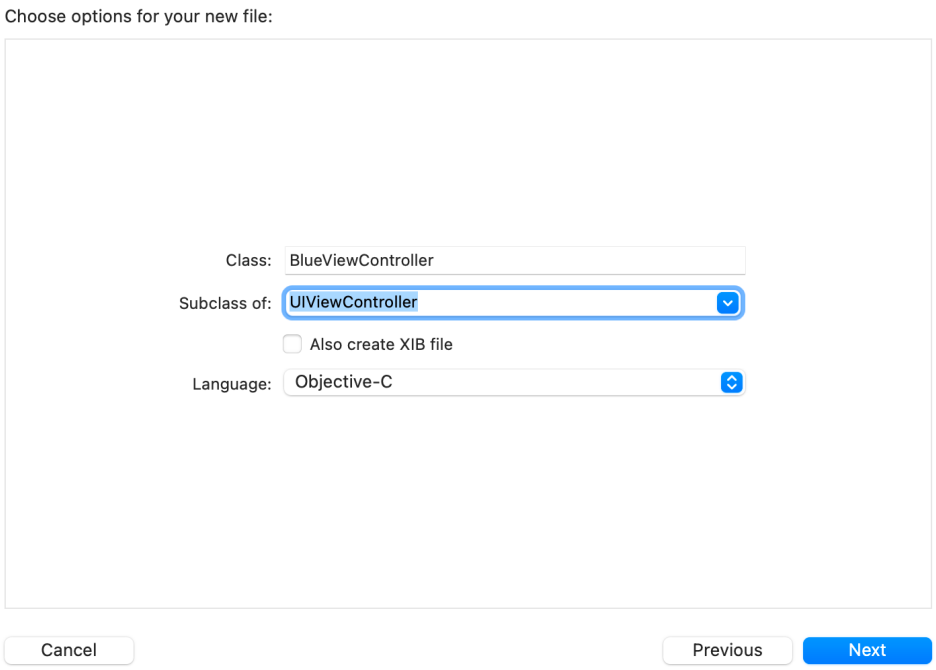
`inspector` (à gauche de l'`Attribute inspector`) que le nom de la classe associé à ce View Controller est bien désormais `OrangeViewController`.



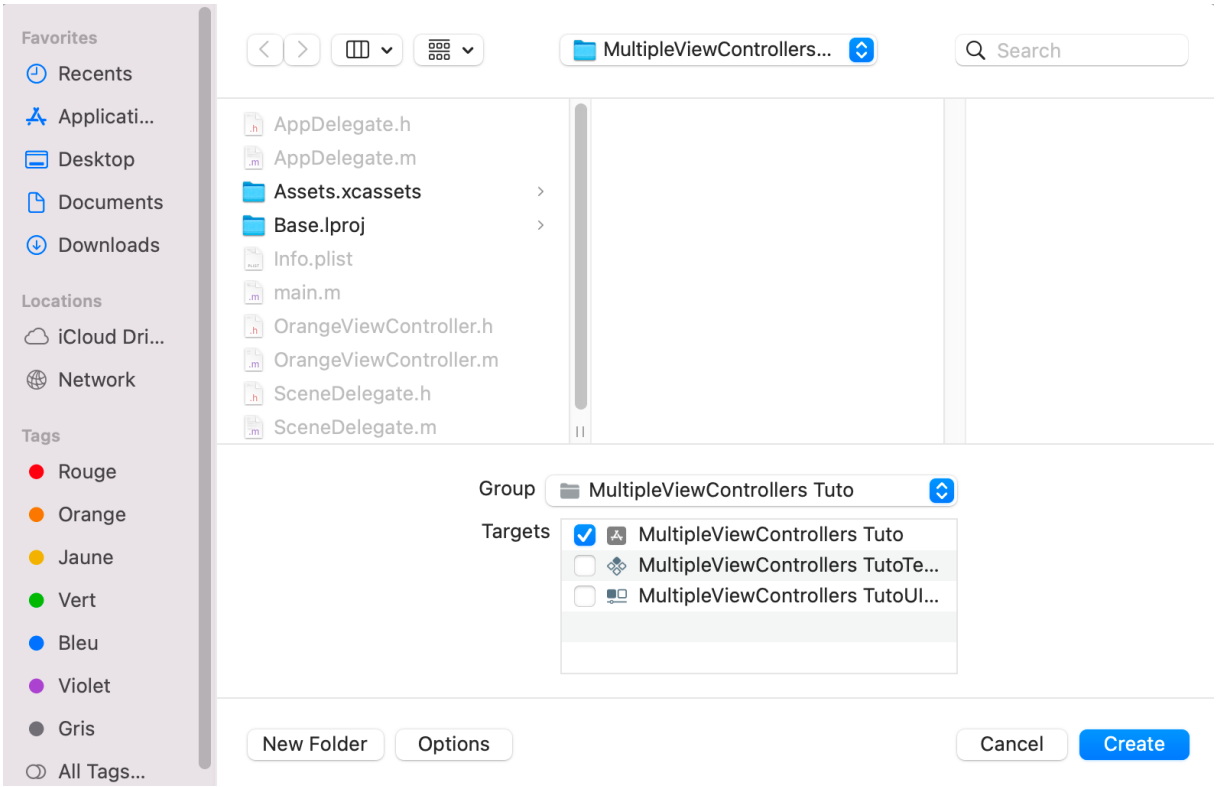
Cela nous permet de constater une petite lacune dans notre beau design : nous n'avons pas de classe pour contrôler l'écran bleu de notre application. Nous allons y remédier. Pour cela faite menu `File > New > File...` et choisissez le modèle `Cocoa Touch Class`.



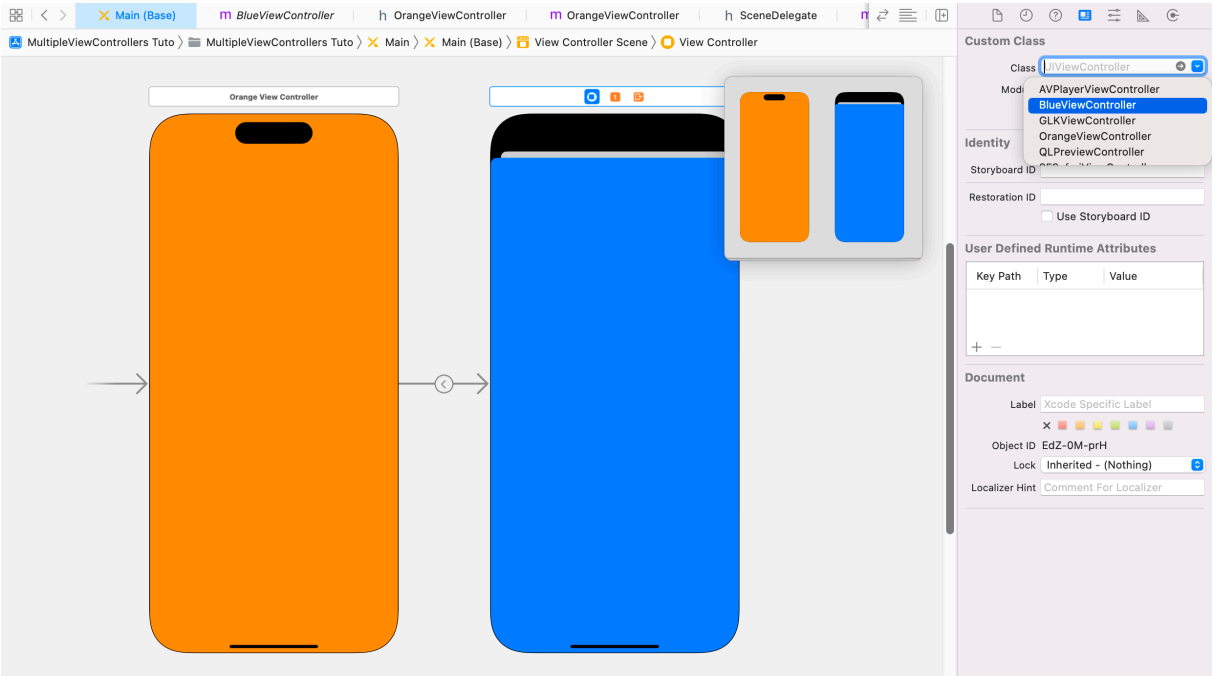
Appelons-là `BlueViewController` et faisons en sorte qu'elle hérite de `UIViewController` (on reste bien entendu en Objective-C).



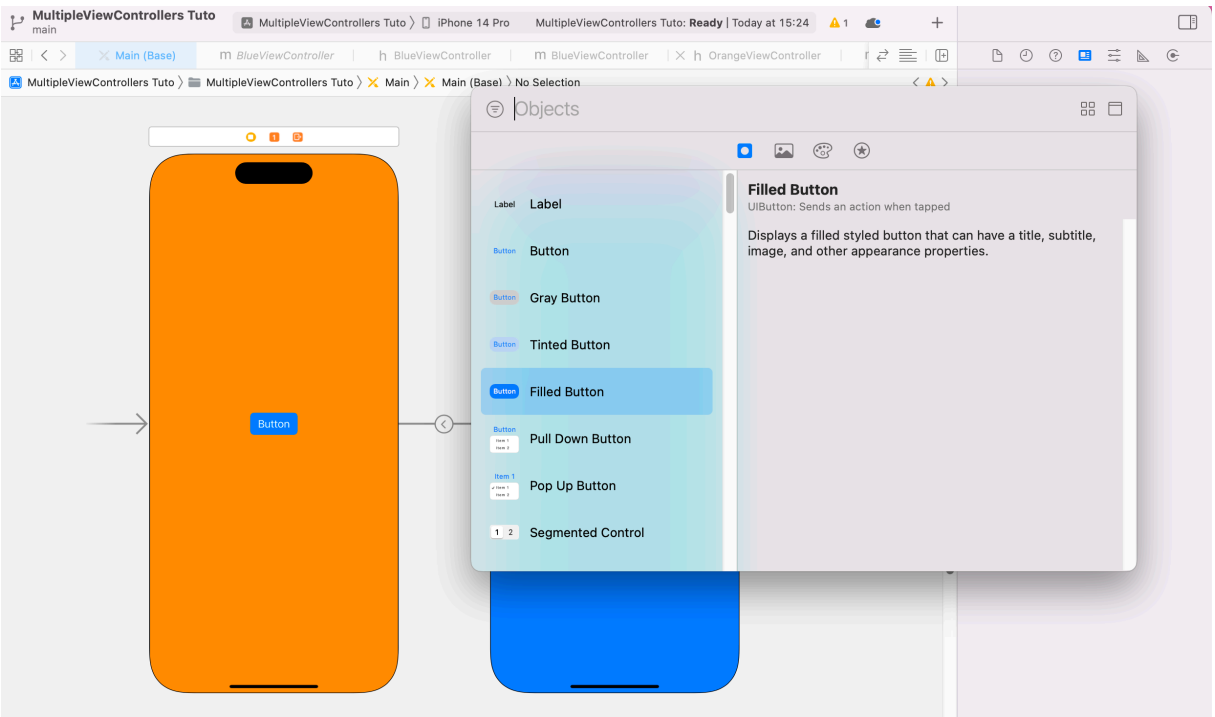
Et on l'ajoute dans le projet.



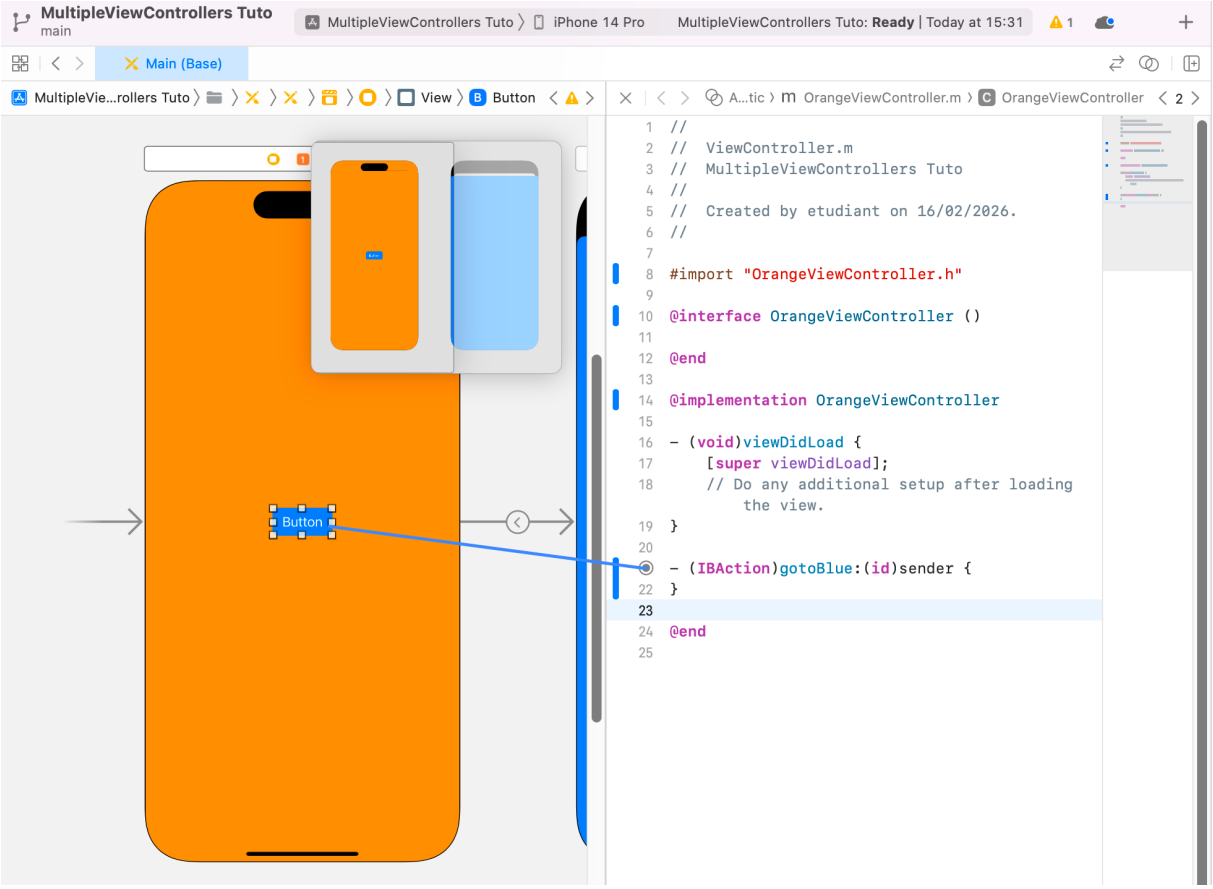
On a ainsi créé deux fichiers `BlueViewController.h` et `BlueViewController.m`. Il ne nous reste plus qu'à associer cette classe `BlueViewController` à la gestion de notre écran bleu. Pour cela, on procède comme quand on a vérifié pour le View Controller orange : on sélectionne, pour ce View Controller bleu, l'icône en forme de disque orange carré blanc, dans le `Main.storyboard`, puis dans l'`Identifier inspector` du panneau latéral droit, on change le nom de la classe par `BlueViewController`.



Afin de déclencher le passage d'un View Controller à l'autre, nous allons poser un bouton sur l'écran orange. On procède comme précédemment : on glisse le bouton depuis la bibliothèque de *widgets* (ici j'ai choisi un `Filled Button`, parce que par défaut il est bleu).



Puis on passe en mode Assistant, on vérifie bien que dans la partie droite de l'écran on a le code de `OrangeViewController.m`, on clique sur le bouton et en maintenant la touche control appuyée on glisse le pointeur de la souris pour ajouter une action (une méthode donc) qu'on appelle ici `gotoBlue`.



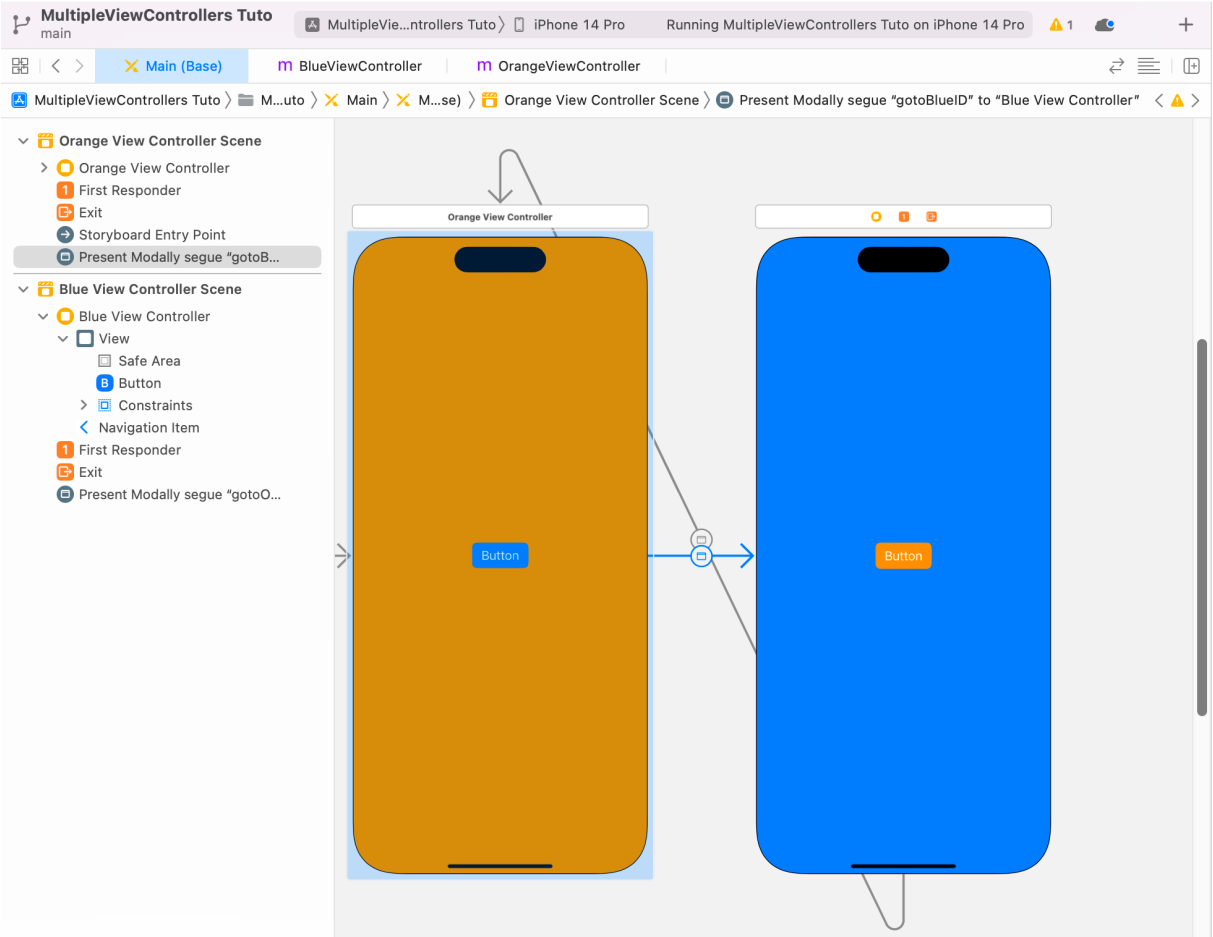
Codons

Il est temps de mettre la main à la pâte. Ajoutons du code à la méthode `gotoBlue:` pour qu'elle devienne :

```
- (IBAction)gotoBlue:(id)sender {
    [self performSegueWithIdentifier:@"gotoBlueID" sender:self];
}
```

Souvenez-vous que nous avions identifiez la transition par : `gotoBlueID`.

Si vous exécutez le programme un clic sur le bouton de l'écran orange fait apparaître l'écran bleu par-dessus (*via* une animation qui le fait monter et se poser sur l'écran orange) comme une carte sur une autre. Il est alors possible de revenir à l'écran orange précédent en faisant glisser le View Controller bleu vers le bas. Néanmoins si on retourne dans le `Main.storyboard`, qu'on sélectionne le segue et que dans l'`Attribute inspector` on choisit `Kind : Present Modally` et `Presentation : Full Screen`, l'écran bleu apparaît de manière modale en plein écran et il n'est alors plus possible de revenir à l'écran orange. La tentation est alors grande de reproduire le même mécanisme pour revenir à l'écran orange : on crée un segue dans l'autre sens, un bouton qui déclenche le retour à l'`OrangeViewController` *via* le `gotoOrangeID` de ce segue.



Ça fonctionne, mais ce n'est pas très joli. On s'en rend compte dans le `Main.storyboard`, le segue prend un chemin bizarre, et quand on exécute l'application, l'animation (par défaut) n'est pas intuitive. En effet, ce n'est pas comme ça qu'on procède en général. On va plutôt dire que le nouveau bouton (orange dans mon cas, parce que j'ai changé sa couleur par défaut) fait disparaître l'écran bleu pour revenir au précédent.

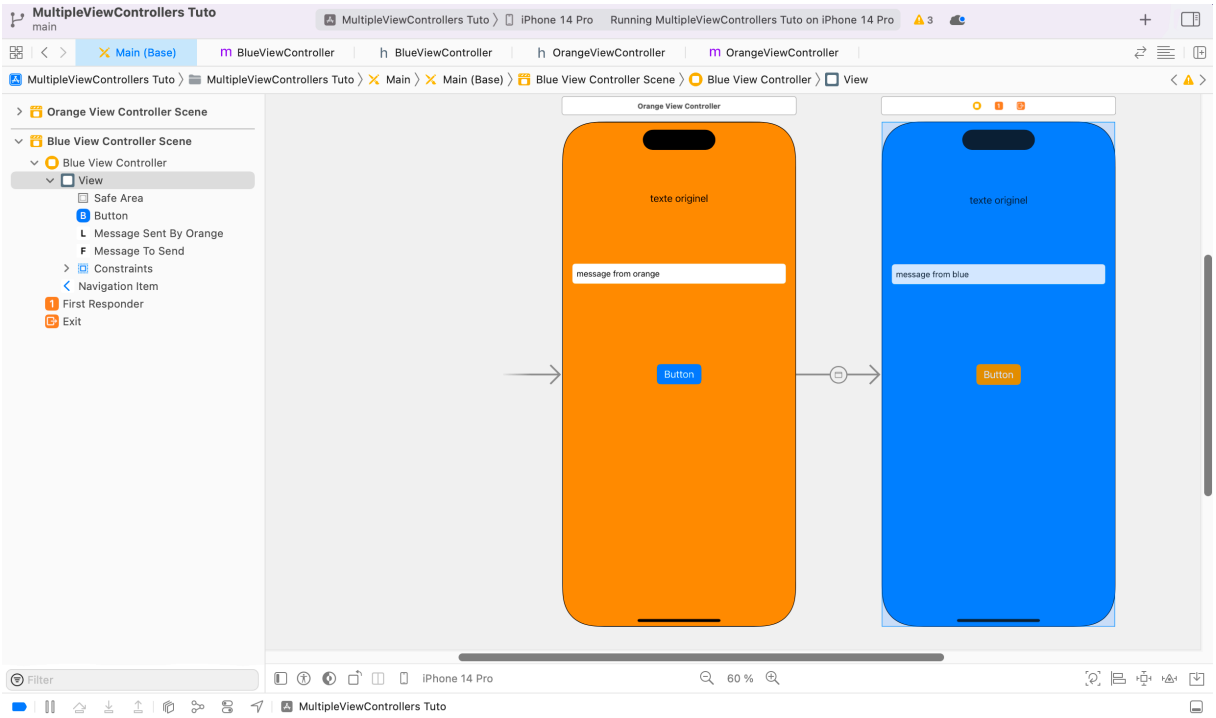
Le code associé à l'action que déclenche le bouton orange devient :

```
- (IBAction)backToOrange:(id)sender {
  //[self performSegueWithIdentifier:@"gotoOrangeID" sender:self];
    [self dismissViewControllerAnimated:YES completion:nil];
}
```

Evolution

Nous avons désormais une application avec deux écrans. Mais nous ne faisons pas transiter d'information entre les deux instances de classe qui leur correspondent. Une première idée pour faire ça consiste à ajouter des attributs à la classe `BlueViewController` (et tant qu'à faire on faire la même chose avec

l'OrangeViewController). On ajoute un Label et un TextField, ça donne le Main.storyboard suivant (on constate qu'on a enlevé le segue de retour bizarre) :



On associe les deux Labels et les deux TextFields aux propriétés correspondantes via des control-glissez dans les ViewController.h respectifs. Ça donne les fichiers suivants :

```
//
//  ViewController.h
//  MultipleViewControllers Tuto
//
toto
#import <UIKit/UIKit.h>

@interface OrangeViewController : UIViewController ;

@property (weak, nonatomic) IBOutlet UILabel *messageSentByBlue;
@property (weak, nonatomic) IBOutlet UITextField *messageToSend;
@property NSString* messageToShow;

@end
```

```
//
//  BlueViewController.h
//  MultipleViewControllers Tuto
//

#import <UIKit/UIKit.h>
```

```
NS_ASSUME_NONNULL_BEGIN

@interface BlueViewController : UIViewController
@property (weak, nonatomic) IBOutlet UILabel *messageSentByOrange;
@property (weak, nonatomic) IBOutlet UITextField *messageToSend;
@property NSString* messageToShow;

@end

NS_ASSUME_NONNULL_END
```

On remarque que quand on a créé la classe `BlueViewController` Xcode nous a généreusement proposé quelques lignes de codes qu'il a placé en commentaires dans `BlueViewController.m` :

```
/*
#pragma mark - Navigation

// In a storyboard-based application, you will often want to do a
little preparation before navigation
- (void)prepareForSegue:(UIStoryboardSegue *)segue sender:(id)sender
{
    // Get the new view controller using [segue
destinationViewController].
    // Pass the selected object to the new view controller.
}
*/
```

On va se servir de ce code pour le placer dans le `OrangeViewController.m`. On va le copier-coller (sans les commentaires) et `OrangeViewController.m` devient :

```
//
//  ViewController.m
//  MultipleViewControllers Tuto
//

#import "OrangeViewController.h"

// ajout obligatoire pour pouvoir connaître la classe BlueController.h
:
#import "BlueViewController.h"

@interface OrangeViewController ()

@end

@implementation OrangeViewController
```

```
- (void)viewDidLoad {
    [super viewDidLoad];
    // Do any additional setup after loading the view.
}

- (IBAction)gotoBlue:(id)sender {
    [self performSegueWithIdentifier:@"gotoBlueID" sender:self];
}

#pragma mark - Navigation

// In a storyboard-based application, you will often want to do a
little preparation before navigation
- (void)prepareForSegue:(UIStoryboardSegue *)segue sender:(id)sender
{
    if ([segue.identifier isEqualToString:@"gotoBlueID"]) {
        // Get the new view controller using [segue
destinationViewController].
        BlueViewController* blueViewController;
        blueViewController = segue.destinationViewController;
        blueViewController.messageToShow = _messageToSend.text;
    }
    // Pass the selected object to the new view controller.
}

@end
```

Il nous suffit maintenant d'aller dans le `BlueViewController.m` pour ajouter le changement de texte au moment où on charge (ou recharge) l'écran bleu. Ainsi le code de `BlueViewController.m` devient :

```
//
//  BlueViewController.m
//  MultipleViewControllers Tuto
//

#import "BlueViewController.h"

@interface BlueViewController ()

@end

@implementation BlueViewController

- (void)viewDidLoad {
    [super viewDidLoad];
    // Do any additional setup after loading the view.
    _messageSentByOrange.text = _messageToShow;;
}
```

```
- (IBAction)backToOrange:(id)sender {
    //[self performSegueWithIdentifier:@"gotoOrangeID" sender:self];
// pas joli !
    [self dismissViewControllerAnimated:YES completion:nil];
}

/*
#pragma mark - Navigation

// In a storyboard-based application, you will often want to do a
little preparation before navigation
- (void)prepareForSegue:(UIStoryboardSegue *)segue sender:(id)sender
{
    // Get the new view controller using [segue
destinationViewController].
    // Pass the selected object to the new view controller.
}
*/

@end
```

On peut tester l'application et se rendre compte que le passage d'information de l'écran orange à l'écran bleu est bien fonctionnel.

On pourrait faire transiter l'information dans l'autre sens en utilsant la même technique. Pour cela il faudrait que BlueViewController possède une propriété de type OrangeViewController (initialisée à self dans le prepareForSegue).
Seulement voilà, se serait faire de la programmation à l'ancienne… c'est-à-dire pas très jolie puisque la classe BlueViewController dépendrait de OrangeViewController. C'est ce qu'on appelle un « couplage fort » (*tight coupling*), ce qui est considéré comme une mauvaise pratique de programmation. En effet, le bleu « possède » l'orange, dans ce cas le bleu ne pourrait plus être réutilisé ailleurs (avec un vert ou un rouge par exemple) sans pour ça être obligé de « casser » le code.

Nous allons mettre en place une solution plus moderne pour faire cheminer l'information dans l'autre sens, c'est-à-dire de l'écran bleu à l'écran orange. Pour cela, nous allons utiliser le motif de conception de délégation, c'est-à-dire le *pattern delegate* (c'est très classique en Objective-C / Cocoa). Le *pattern delegate* consiste à confier une partie de ses responsabilités à un autre objet. Imaginez une relation maître-assistant : l'objet déléguant (le maître) a une tâche à faire mais ne sait pas ou ne veut pas gérer certains détails ou décisions ; tandis que le délégué (l'assistant) écoute le maître et exécute ses ordres ou lui fournit les informations qu'il demande. Concrètement dans le cadre de notre application il sert à faire transiter de l'information entre les deux instances de classe.

Dans notre cas : l'`OrangeViewController` va préparer le passage vers le bleu; il se définit lui-même comme le délégué du `BlueViewController`, quand l'utilisateur appuie sur le bouton, au lieu de simplement disparaître le `blueViewController` appelle une méthode chez son délégué pour lui transmettre des informations.

D'abord, il faut déclarer le protocol de délégation pour le `BlueViewController`. Dans le `BlueViewController.h` on ajoute (après les #import) :

```
@protocol BlueViewControllerDelegate <NSObject>
- (void) blueViewControllerDidReturnMessage:(NSString *) message;
@end
```

Dans l'`@interface`, on ajoute également une nouvelle propriété :

```
@property (nonatomic, weak) id<BlueViewControllerDelegate> delegate;
```

Dans le `BlueViewController.m` c'est dans la méthode `backToOrange:` que le `BlueViewController` dit à son délégué qu'il doit utiliser la méthode `blueViewControllerDidReturnMessage:` on a donc :

```
//
//  BlueViewController.h
//  MultipleViewControllers Tuto
//

#import <UIKit/UIKit.h>

NS_ASSUME_NONNULL_BEGIN

@protocol BlueViewControllerDelegate <NSObject>
- (void) blueViewControllerDidReturnMessage:(NSString *) message;
@end

@interface BlueViewController : UIViewController
@property (weak, nonatomic) IBOutlet UILabel *messageSentByOrange;
@property (weak, nonatomic) IBOutlet UITextField *messageToSend;
@property NSString* messageToShow;
@property (nonatomic, weak) id <BlueViewControllerDelegate> delegate;
@end

NS_ASSUME_NONNULL_END
```

```
//
//  BlueViewController.m
//  MultipleViewControllers Tuto
//

#import "BlueViewController.h"

@interface BlueViewController ()

@end

@implementation BlueViewController
```

```
- (void)viewDidLoad {
    [super viewDidLoad];
    // Do any additional setup after loading the view.
    _messageSentByOrange.text = _messageToShow;
}
- (IBAction)backToOrange:(id)sender {
    [self.delegate
blueViewControllerDidReturnMessage:_messageToSend.text];
    [self dismissViewControllerAnimated:YES completion:nil];
}

@end
```

Côté OrangeViewController, il faut mettre en place le délégué et implémenter le protocole :

```
//
//  ViewController.h
//  MultipleViewControllers Tuto
//

#import <UIKit/UIKit.h>
#import "BlueViewController.h"

@interface                    OrangeViewController                    :
UIViewController<BlueViewControllerDelegate> ; // changement ici !

@property (weak, nonatomic) IBOutlet UILabel *messageSentByBlue;
@property (weak, nonatomic) IBOutlet UITextField *messageToSend;
@property NSString* messageToShow;

@end
```

```
//
//  ViewController.m
//  MultipleViewControllers Tuto
//

#import "OrangeViewController.h"

// ajout obligatoire pour pouvoir connaître la classe BlueController.h
:
#import "BlueViewController.h"

@interface OrangeViewController ()

@end
```

```
@implementation OrangeViewController

- (void)viewDidLoad {
    [super viewDidLoad];
    // Do any additional setup after loading the view.
}

- (IBAction)gotoBlue:(id)sender {
    [self performSegueWithIdentifier:@"gotoBlueID" sender:self];
}

#pragma mark - Navigation

// In a storyboard-based application, you will often want to do a
little preparation before navigation
- (void)prepareForSegue:(UIStoryboardSegue *)segue sender:(id)sender
{
    if ([segue.identifier isEqualToString:@"gotoBlueID"]) {
        // Get the new view controller using [segue
destinationViewController].
        BlueViewController* blueViewController;
        blueViewController = segue.destinationViewController;
        blueViewController.messageToShow = _messageToSend.text;
        blueViewController.delegate = self;  // on se positionne comme
le délégué du bleu
    }
    // Pass the selected object to the new view controller.
}



- (void)blueViewControllerDidReturnMessage:(nonnullNSString *)message
{
    _messageSentByBlue.text = message;
}


@end
```

Il est inutile de créer un delegate pour l'aller de l'orange vers le bleu, car `prepareForSegue` nous donne déjà un accès direct à l'objet de destination. Le delegate est l'outil spécifique pour remonter le flux d'information.